Guía de Actividades **PrácticoExperimentales** Nro. 003

1. Datos Generales

Asignatura	Diseño Digital
Ciclo	5to
Unidad	Uno
Resultado de aprendizaje de la unidad	R1. Diseña arquitectura de microcontroladores, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	3
Título de la Práctica	Temporizador en Arduino + Generación y Visualización de Señal
Nombre del Docente	José O. Guamán Q.
Nombre del Estudiante	Viviana Elizabeth Córdova Celi
Fecha	21-04-2026 23-04-2026
Horario	Martes 12h30 – 13h30 jueves 11h30 – 13h30
Lugar	Aula, laboratorio
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

- Diseñar un sistema híbrido que genere, transforme y mida señales digitales usando
- un NE555 Timer IC, un 74HC74 y Arduino como analizador
- Diseñar un oscilador estable con 555
- Corregir duty cycle mediante flip-flop
- Implementar división de frecuencia
- Medir frecuencia y periodo con Arduino (sin delay/interrupciones)
- Analizar error, estabilidad y jitter

3. Materiales y reactivos:

NE555 Timer IC
74HC74
Arduino Uno
2 LEDs + resistencias
Resistencias: 1kΩ, 10kΩ, 100kΩ
Capacitores: 1µF, 10µF
Protoboard + jumpers **Problema real**

El 555 en astable:

Tiene duty cycle > 50%
No es perfectamente estable
Tiene tolerancias

- **Solución**
- Flip-flop → corrige duty a ~50%
- Arduino → mide señal y calcula parámetros

5. Equipos y herramientas

- Computadora con Arduino IDE
- Componentes electrónicos
- Protoboard
- Serial Monitor / Plotter

6. Procedimiento / Metodología

FASE 1: Generación de señal (hardware)

- Configurar el NE555 Timer IC en modo astable
- Ajustar R1, R2 y C para obtener una frecuencia observable (≈ 1–10 Hz)
- Conectar LED a la salida para validación visual

Resultado: señal periódica base (no ideal, duty > 50%)

Pin 555	Conexión
Pin 1	GND
Pin 8	VCC (5V)
Pin 4 (RESET)	VCC
Pin 2 (TRIG)	Unir con pin 6
Pin 6 (THRESH)	Unir con pin 2
Pin 7 (DISCH)	Nodo entre R1 y R2
Pin 3 (OUT)	**Salida principal (señal)**
Pin 5	(Opcional) capacitor 10nF a GND

Especificación y modelado

Definición de requisitos:

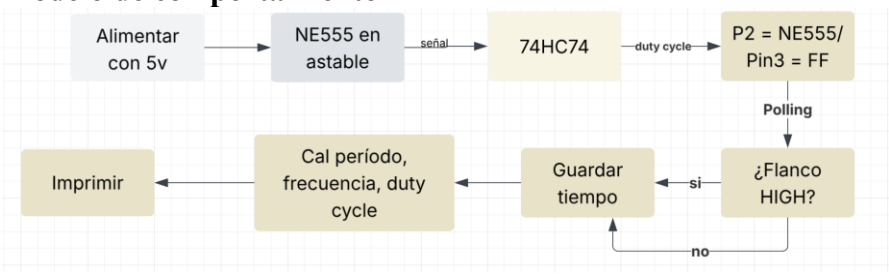
La implementación comienza con la configuración del circuito integrado NE555 en modo astable. El objetivo primordial es generar una oscilación autónoma mediante la selección estratégica de componentes pasivos: las resistencias $R1$, $R2$ y el capacitor C se ajustan para producir una frecuencia baja y observable, situada en el rango de 1 a 10 Hz.

Funcionalidades:
El circuito debe cumplir con las siguientes funciones
- Generar una señal cuadrada periódica usando el NE555 Timer IC.
- Permitir ajuste de frecuencia mediante R1, R2 y C.
- Proporcionar una salida visible mediante LED.
- Entregar señal base para procesamiento posterior.

Restricciones de tiempo
- La frecuencia debe estar entre **1 Hz y 10 Hz** para facilitar observación visual.
- La señal debe mantenerse continua y estable.

Modelado del sistema:

Diagramas UML
Modelo de comportamiento



- **Máquinas de estados finitos**

El sistema funciona mediante estados secuenciales. Primero se inicializa los componentes y configurando el Arduino. Luego el NE555 genera una señal periódica, la cual pasa al Flip-Flop 74HC74 para corregir su ciclo de trabajo y dividir la frecuencia. Después, el Arduino entra en estado de lectura continua, detecta cambios de LOW a HIGH, procesa los tiempos medidos y finalmente muestra los resultados en pantalla. Este ciclo se repite constantemente mientras el sistema esté encendido.

- **Definir Funcionalidades**

El sistema tiene como función principal generar, transformar y medir señales digitales. El NE555 produce los pulsos iniciales, el 74HC74 mejora la señal obteniendo una salida más estable, y el Arduino captura los datos de ambas señales para calcular frecuencia, período y duty cycle. Cuando se trata de datos, el sistema los adquiere en tiempo real, los procesa inmediatamente y los envía al monitor serial para su visualización y análisis continuo.

FASE 2: Corrección y transformación

- Conectar salida del 555 al CLK del 74HC74
- Configurar: D = NOT(Q)
- Obtener salida en Q Resultado:

- Frecuencia dividida entre 2
- Duty cycle ≈ 50%

Pin 74HC74	Conexión
Pin 14	VCC (5V)
Pin 7	GND
Pin 3 (CLK)	← Pin 3 del 555
Pin 2 (D)	← NOT(Q)
Pin 5 (Q)	**Salida FF**
Pin 6 (NOT Q)	→ conectar a D
Pin 1 (CLR)	VCC
Pin 4 (PRE)	VCC

Diseño de hardware y software

Hardware

El hardware del sistema está compuesto por una arquitectura distribuida en tres módulos principales:

- El NE555 como generador de señal periódicas en modo astable.
- El 74HC74 como corrector y divisor de frecuencia de la señal generada por 555.
- El Arduino UNO como unidad de adquisición y procesamiento.
- En cuanto a memoria, el Arduino utiliza memoria interna para almacenar variables
- temporales y resultados de medición.
- Los periféricos de entrada/salida corresponden a los pines digitales que reciben las señales
- del NE555 y del Flip-Flop, mientras que la salida de datos se realiza mediante el puerto
- serial hacia el computador.
-

Software

El software fue desarrollado en lenguaje Arduino C/C++, adecuado para sistemas embebidos por su control directo del hardware. No se utilizó un RTOS, ya que el sistema trabaja con una sola tarea repetitiva de lectura y procesamiento continuo. Se priorizó la eficiencia mediante un programa sin uso de delay(), permitiendo respuesta rápida en tiempo real. Además, el código fue organizado de forma modular, separando funciones de lectura, cálculo y visualización para facilitar mantenimiento y futuras mejoras.

Cocreación.

- Detectar flancos ascendentes.
- Medir período y frecuencia.
- Calcular el duty cycle.
- Procesar dos señales de forma simultánea.
- Mostrar resultados mediante comunicación serial.

FASE 3: Adquisición de señal con Arduino

Conectar:

Salida 555 → Pin 2 Arduino

Salida Flip-Flop → Pin 3 Arduino

Configurar Arduino como sistema de adquisición

Leer continuamente señales digitales (polling) Sin usar: delay() interrupciones

Señal	Pin Arduino
Salida 555 (Pin 3)	Pin 2
Salida Flip-Flop (Pin 5)	Pin 3
GND común	GND Arduino

Dentro de la Fase 3 se implementó el Arduino UNO como un sistema que permita la adquisición de datos digitales. Este se encarga de captar, procesar y visualizar ñas señales generados por el circuito electrónico. Prara ello, la salida del temporizador NE555 se conecto al pin digital 2 del Arduino, mientras que la salida del flip-flop se conecto al pin digital 3, lo que permitió leer los dos canales y los dos resultados.

Implementación e integración
La adquisición se realizó mediante la técnica de **polling**, es decir, el microcontrolador consulta continuamente el estado lógico de cada pin sin utilizar funciones delay() ni interrupciones, logrando una lectura rápida y estable en tiempo real. Esto permitió detectar cambios de estado (HIGH/LOW), medir periodos y frecuencias de las señales, y representar los resultados en el monitor serial.
Grabado
El programa se cargó al Arduino mediante conexión USB utilizando al Arduino IDE.
Durante este proceso:

- Se asignaron los puertos p2 y p3
- Se cargo el código al IDE
- Se cargo a la placa

Después de cargarse el sistema comenzó a ejecutarse automáticamente.

Integración de plataforma
Memoria:
El sistema dispone de memoria interna utilizada para almacenar el programa cargado, variables de trabajo y datos temporales durante la ejecución.
Periféricos:

- Pines digitales: para leer la salida del NE555 y del Flip-Flop.
- Puerto serial: para enviar resultados al computador.
- Temporizadores internos: para medir tiempos y frecuencias.
- LEDs indicadores: para validación visual de estados.

CPU en PCB
La CPU corresponde al microcontrolador integrado en la placa (**ATmega328P**), encargado de ejecutar instrucciones, procesar datos y controlar los periféricos, y al estar montado en la PCB, este coordina las operaciones de adquisición y análisis en tiempo real.

FASE 4: Procesamiento de señal (software) Para

cada señal:

- Detectar flancos ascendentes
  Para detectar los flancos ascendentes de la señal, se comparó de forma continua el estado actual con el estado anterior de cada una de las entradas digitales.
  Cuando un flanco asciende ocurre la señal cambia de LOW(0) a HIGH(1)
  Para la detección de los flancos ascendentes de la señal
- Medir tiempo entre flancos
- Calcular:

  Frecuencia y Periodo del Flip-Flop

  Medir tiempos HIGH y LOW

  Los tiempos HIGH y LOW se obtienen registrando el instante en que la señal cambia de estado usando el tiempo del sistema (millis()). Cada vez que ocurre un cambio se calcula diferencia entre el tiempo actual y el ultimo cambio. De modo que, si se mide de forma continua los cambios, se puede llegar a determinar

  Calcular duty cycle

  Para cada señal capturada, el software ejecuta una detección de flancos ascendentes comparando estados lógicos consecutivos. Una vez identificado el flanco, se emplea la función millis() para medir el intervalo temporal entre eventos, lo que permite calcular con precisión el periodo y, consecuentemente, la frecuencia mediante la relación $f = 1000 / T$.

FASE 5: Visualización

- Mostrar resultados en Serial Monitor
- Graficar señal en Serial Plotter • Comparar:

  Señal 555 vs Flip-Flop

  En la fase de visualización se logró representar el comportamiento de las señales generadas por el NE555 y el flip-flop mediante el uso del Serial Monitor y el Serial Plotter de Arduino, lo que permitió observar en tiempo real los cambios de estado y la forma de onda digital de ambas señales. A través de esta visualización se facilitó la comparación directa entre la señal original del 555 y la señal dividida por el 74HC74, evidenciando la relación de frecuencia entre ambas

FASE 6: Análisis experimental

- Comparar valores teóricos vs medidos
- Calcular error
- Evaluar estabilidad (jitter)
- Interpretar diferencias

	Valores Simulados		**Valores reales**	
	555	**Flip-flop**	**555**	**Fli-flop**
Frecuencia	94.05Hz	6.17Hz	4.93Hz	2.43 Hz
Periodo	84ms	162ms	203 ms	411ms
Duty	94.05%	48.15 %	50.61 %	50.61 %

Calculo de error

Frecuencia

555

$$Error = | 94.05 - 4.93 \ / \ 4.93 | \ \times 100$$

$$Error = \frac{89.12}{4.93} \times 100 \approx 1807.9\%$$

flip -flop

$$Error = | 6.17 - 2.43/2.43 | \ \times 100$$

$$Error = \frac{3.74}{2.43} \times 100 \approx 153.9\%$$

Periodo

555

$$Error = \frac{119}{203} \times 100 \approx 58.6\%$$

Flip-flop

$$Error = \frac{249}{411} \times 100 \approx 60.6\%$$

Duty Timer

555

$$Error = \frac{43.44}{50.61} \times 100 \approx 85.9\%$$

Flip-flop

$$Error = \frac{2.46}{50.61} \times 100 \approx 4.86\%$$

Análisis de resultados

Los resultados obtenidos cumplen parcialmente el comportamiento esperado del sistema. En el NE555, la frecuencia se mantiene cercana a la calculada teóricamente, aunque con desviaciones propias de la simulación y la tolerancia de los componentes; además, el duty cycle es mayor al 50%, lo cual es característico del modo astable utilizado. En el flip-flop 74HC74, se observa que la frecuencia se reduce aproximadamente a la mitad de la señal original del 555 y el duty cycle tiende a estabilizarse cerca del 50%, cumpliendo su función de división de frecuencia y simetrización de la señal. En el Serial Plotter de Arduino, se visualizan dos ondas cuadradas simultáneas con diferentes frecuencias, lo que confirma el correcto procesamiento y medición de ambas señales mediante polling. Finalmente, el código generado debe ser subido a GitHub como evidencia del desarrollo y validación del sistema.

Algoritmos

Algoritmo 1: Detección de flanco (Edge Detection)

Leer estado actual
Si (estado actual == HIGH) y (estado anterior == LOW):
  Detectar flanco ascendente
Guardar estado actual como anterior

Algoritmo 2: Medición de periodo

Cuando ocurre flanco ascendente:
  tiempoActual = millis() periodo = tiempoActual - tiempoAnterior
tiempoAnterior = tiempoActual **Algoritmo 3: Cálculo de frecuencia** frecuencia = 1000 / periodo **Algoritmo 4: Medición de duty cycle**

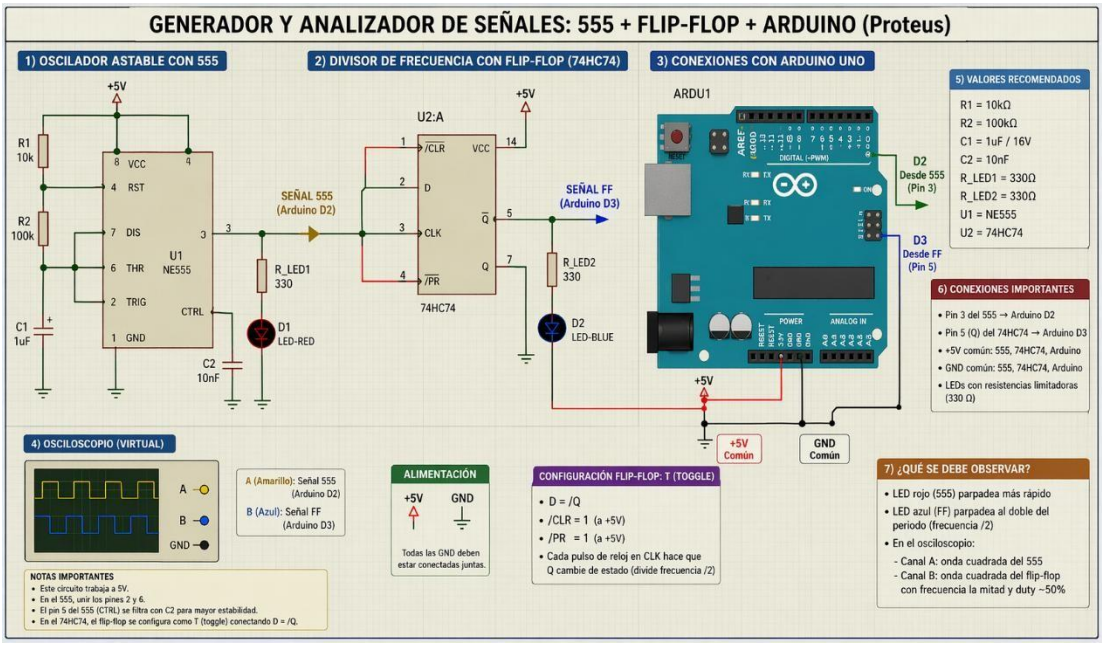
Si cambia estado: Si nuevo estado es HIGH:
    tLow = tiempoActual - tiempoCambio
Si nuevo estado es LOW:
    tHigh = tiempoActual - tiempoCambio

  tiempoCambio = tiempoActual

duty = tHigh / (tHigh + tLow) * 100

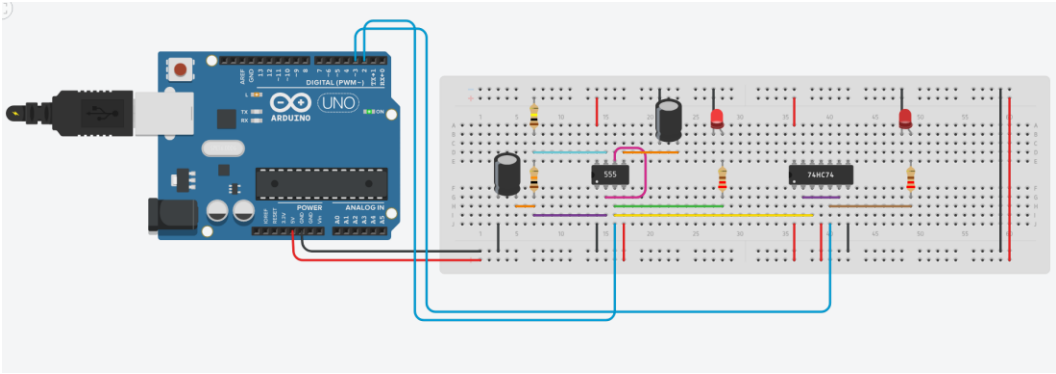
Algoritmo 5: Multicanal (dos señales) Repetir todos los algoritmos para:
  Canal 1 (555)
  Canal 2 (Flip-Flop)

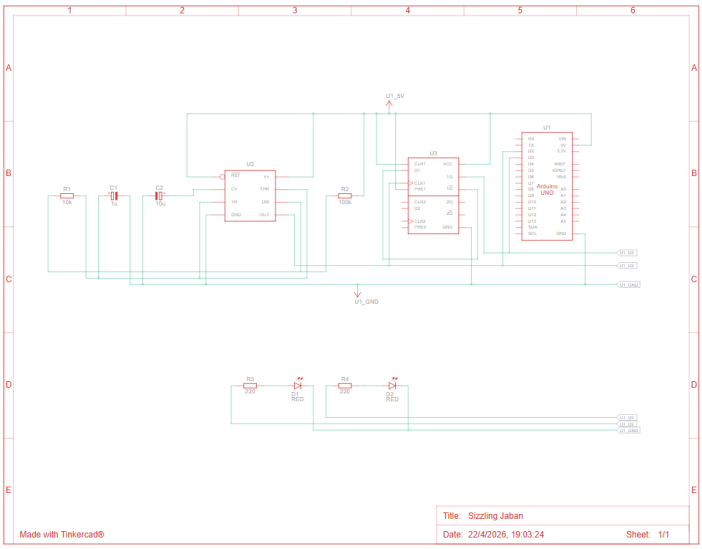
DIAGRAMA A CONCIDERAR



6. Resultados esperados:

555:
Frecuencia ≈ calculada
Duty > 50%
Flip-Flop:
Frecuencia ≈ mitad
Duty ≈ 50%
Serial Plotter:
Dos ondas cuadradas
Diferente frecuencia
Subir el código
generado en Git
El proyecto generado
en forma virtual
El proyecto debe seguir
los pasos de la
diapositiva de la semana
2
proyectos embebidos
Proyecto funcional





Código

```
const int pin555 = 2;
const int pinFF  = 3;

// Variables canal 1
int estadoActual1 = LOW;
int estadoAnterior1 = LOW;

unsigned long tiempoAnterior1 = 0;
unsigned long periodo1 = 0;

unsigned long tiempoCambio1 = 0;
unsigned long tHigh1 = 0;
unsigned long tLow1 = 0;

// Variables canal 2
int estadoActual2 = LOW;
int estadoAnterior2 = LOW;

unsigned long tiempoAnterior2 = 0;
```

```
unsigned long periodo2 = 0;

unsigned long tiempoCambio2 = 0;
unsigned long tHigh2 = 0;
unsigned long tLow2 = 0;

// Control impresión
unsigned long ultimoPrint = 0;

//=====================================================
void setup() {

  pinMode(pin555, INPUT);
  pinMode(pinFF, INPUT);

  Serial.begin(9600);
}

//=====================================================
void loop() {

  unsigned long tiempoActual = millis();

  //==================================
  // CANAL 555
  //==================================
  estadoActual1 = digitalRead(pin555);

  if (estadoActual1 == HIGH && estadoAnterior1 == LOW) {
    periodo1 = tiempoActual - tiempoAnterior1;
    tiempoAnterior1 = tiempoActual;
  }

  if (estadoActual1 != estadoAnterior1) {

    if (estadoActual1 == HIGH)
      tLow1 = tiempoActual - tiempoCambio1;
    else
      tHigh1 = tiempoActual - tiempoCambio1;

    tiempoCambio1 = tiempoActual;
  }

  estadoAnterior1 = estadoActual1;
```

```
//==================================
// CANAL FLIP FLOP
//==================================
estadoActual2 = digitalRead(pinFF);

if (estadoActual2 == HIGH && estadoAnterior2 == LOW) {
  periodo2 = tiempoActual - tiempoAnterior2;
  tiempoAnterior2 = tiempoActual;
}

if (estadoActual2 != estadoAnterior2) {

  if (estadoActual2 == HIGH)
    tLow2 = tiempoActual - tiempoCambio2;
  else
    tHigh2 = tiempoActual - tiempoCambio2;

  tiempoCambio2 = tiempoActual;
}

estadoAnterior2 = estadoActual2;

//==================================
// CALCULOS
//==================================
float freq1 = 0;
float duty1 = 0;

if (periodo1 > 0)
  freq1 = 1000.0 / periodo1;

if ((tHigh1 + tLow1) > 0)
  duty1 = (float)tHigh1 / (tHigh1 + tLow1) * 100.0;

float freq2 = 0;
float duty2 = 0;

if (periodo2 > 0)
  freq2 = 1000.0 / periodo2;

if ((tHigh2 + tLow2) > 0)
  duty2 = (float)tHigh2 / (tHigh2 + tLow2) * 100.0;

//==================================
// IMPRESION CADA 1 SEGUNDO
```

```
  //==================================
  if (tiempoActual - ultimoPrint >= 1000) {

    ultimoPrint = tiempoActual;

    Serial.println();
    Serial.println("====== SENAL 555 ======");
    Serial.print("Periodo (ms): ");
    Serial.println(periodo1);

    Serial.print("Frecuencia (Hz): ");
    Serial.println(freq1);

    Serial.print("Duty cycle(%): ");
    Serial.println(duty1);

    Serial.println();

    Serial.println("====== FLIP-FLOP ======");
    Serial.print("Periodo (ms): ");
    Serial.println(periodo2);

    Serial.print("Frecuencia (Hz): ");
    Serial.println(freq2);

    Serial.print("Duty cycle(%): ");
    Serial.println(duty2);

    Serial.println();
    Serial.println("------------------------");
    Serial.println();
  }

  //==================================
  // SERIAL PLOTTER
  // IMAGEN
  //==================================
  Serial.print(estadoActual1);
  Serial.print(",");
  Serial.println(estadoActual2);
}
```

Link Tinkercard: https://www.tinkercad.com/things/3cgWd4GHyos-555ape3
Link Gib-Hub:
https://github.com/vivinaCordova/Arquitectura_Microcontroladores.git

7. Declaración del uso de IA

Se utilizó inteligencia artificial generativa, incluyendo herramientas como NotebookLM, a la cual se le cargaron los documentos y materiales de apoyo de la práctica. Esta herramienta permitió resolver dudas puntuales y reforzar la comprensión de los conceptos relacionados con la generación y medición de señales, facilitando el desarrollo de las preguntas de control y el análisis de resultados.

Adicionalmente, se utilizó ChatGPT y Claude AI como apoyo para la generación y depuración del código utilizado en la práctica, así como para la organización y redacción del informe. En todo momento se mantuvo un criterio crítico frente a la información generada, verificando los resultados con las mediciones y simulaciones realizadas.

8. Preguntas de Control:

¿El 555 genera señal, pero con limitaciones?

El temporizador NE555 en modo astable puede generar una señal periódica estable (onda cuadrada), pero su precisión no es absoluta. Sus principales limitaciones son la tolerancia de los componentes (resistencias y capacitores), la deriva térmica y la falta de control fino en la estabilidad de frecuencia a largo plazo. Por eso, la señal puede variar ligeramente respecto a los valores teóricos.

¿El flip-flop corrige y estabiliza?

El flip-flop (como el 74HC74) actúa como un elemento de sincronización y división de frecuencia. Su función principal es limpiar la señal del 555, eliminando rebotes o variaciones pequeñas, y generar transiciones más definidas (estados digitales estables 0 y 1). Esto mejora la calidad de la señal para su lectura en sistemas digitales como Arduino.

¿El método de polling permite medición sin interrupciones?

El polling consiste en que el microcontrolador consulta constantemente el estado de un pin de entrada dentro del loop principal. Esto permite medir eventos como flancos de subida o bajada sin usar interrupciones externas. Aunque es más simple de implementar, puede perder precisión si el programa está ocupado en otras tareas o si la señal es muy rápida.

¿La precisión depende del tiempo de muestreo (millis)?

La precisión de las mediciones depende del intervalo de muestreo usado con funciones como millis(). Mientras menor sea el tiempo entre lecturas, mayor será la resolución para detectar cambios en la señal. Sin embargo, si el muestreo es demasiado lento, se pueden perder transiciones; y si es muy rápido, puede aumentar la carga de procesamiento sin mejorar significativamente la precisión.

9. Evaluación

Criterio	Excelente	Bueno	Bajo
Montaje y funcionamiento	Funciona completo (4)	Parcial (2)	No funciona (0)
Funcionamiento virtual	Funciona completo (3)	Parcial (2)	No funciona (0)
Documento	Todos los puntos (2)	Parcial (1)	No valido (0)
Defensa	1	0,5	0

10. Bibliografía

- Semana 2. Guaman, José. 2026
- Semana 3. Guaman, José. 2026
- Datasheet NE555
- Datasheet 74HC74

11. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	
Aprobado por	Edison L Coronel Romero **Director de Carrera**	